

Methods of the Worker

_Methods of the Worker

The *Worker* provides:

- The *methods* listed in the table of contents to the left.

Note

If you do not use a [FootPath](#), *Plant Simulation* beams/teleports the *Worker* to the *Workplace*.

- The [_Methods of the Exporter](#) as the *Worker* is an [Exporter \[object\]](#) with the capacity 1.
- The [Methods of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

Figure 1: Methods of the Worker (Station)

Name	Value	Inherited	Watchable	Signature
contentsList	[]			(([Contents:table]) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				((IconName:string, Width:integer, Height:integer) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of the method Identifier and data type of the parameter you enter Data type of the return value

Figure 2: Methods of the Worker (Station)

Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			((Index:integer:=1) -> object
MUPart	VOID			((Index:integer:=1) -> object

Name of the method Identifier and data type of the parameter you enter Default value Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, *:= false* in the example above.

If the *method* has a return value, the signature shows its data type after the arrow *->*, *→ boolean* in the example above.

cancelTransportPart [SimTalk]

Cancels each and every transport of the parts, which you started with the method *transportPart*, and which are scheduled to be carried by the *Worker* designated by <Path>.

Remarks

If one part or several parts are scheduled to be transported, *Plant Simulation* deletes this reservation and the *Worker* either walks to the *WorkerPool* or to his base.

If you specified a *Station* at which the *Worker* picks up finished parts, *Plant Simulation* deletes this reservation as well. As soon as the *Worker* has picked up a part and transported it, the reservation cannot be canceled though.

Note

If you specified a **Destination** when ordering the *Worker* to transport parts, and the **Destination** was then saved in the parts to be transported, *Plant Simulation* does not undo this change. This means that the specified **Destination** is retained and you can still query it with the attribute *Destination*.

Note

cancelTransportPart only cancels each and every transport of the parts, which you started with the method *transportPart*.

It does not cancel automatically mediated transports initiated by a combination of the *Transport Importer* and the *Broker*.

Type

Method

Syntax

```
<Path>.cancelTransportPart -> boolean
```

Return Value

The return value has the data type *boolean*.

true if all transport processes could be canceled.

false if they could not be canceled.

false might be returned if no orders exist to transport parts or if the *Worker* picked up at least one part to be transported.

Example

```
.Resources.MyWorker:1.cancelTransportPart
```

SimTalk

[transportPart \[SimTalk\]](#)

[Destination \[SimTalk\] - MUs](#)

create [SimTalk] - Worker

Creates an instance of the *Worker* designated by <Path> in the *WorkerPool*.

Type

Method

Syntax

```
<Path>.create(WorkerPool:object)
```

Parameter

The parameter *WorkerPool* of data type *object* designates the *WorkerPool* where the *Worker* will be created.

Example

```
.Resources.MyWorker.create(MyWorkerPool)
```

getRouteLength [SimTalk] - Worker

Returns the route of the *Worker* designated by <Path> from his present location to this destination object.

Remarks

If you enter a station as the destination object, the *method* returns the length of the route to the *Workplace* which can be reached on the shortest route.

If the *Worker* is exporting a service at the moment, it only takes those *Workplaces* into consideration, which support the service, including all *Workplaces*, which are not restricted to certain services.

If you enter a *Workplace* as destination object, *Plant Simulation* always computes the length to this *Workplace*, no matter if it supports the exported service or not.

Type

Method

Syntax

```
<Path>.getRouteLength(Target:path[, byref FoundWorkplace:object]) → length
```

Parameters

You can specify the following parameters:

- The parameter *Target* of data type *path* designates the destination object. The destination object can be a *Workplace*, a *WorkerPool*, or a *work station* to which you attached *Workplaces*.
- The optional local variable *FoundWorkplace* of data type *object* is the variable into which *Plant Simulation* writes the *Workplace* which it found at the destination object.

This is helpful if you assign a *work station* to the parameter *Target* instead of a *Workplace*.

Return Value

The return value has the data type *length*.

Is -1 if *Plant Simulation* did not find a route.

Examples

```
print .Resources.Worker:1.getRouteLength(MyWorkplace1)
```

```
var FoundWorkplace: object  
.Resources.Worker:1.getRouteLength(MyStation, FoundWorkplace)  
print FoundWorkplace
```

SimTalk

[getRouteCoordinates \[SimTalk\]](#)

[getRouteLength \[SimTalk\] - Workplace](#)

See also

[Route to Destination \[Worker\]](#)

getServices [SimTalk] - Worker

Returns all services of the *Worker* designated by <Path> and writes them into a table.

Type

Method

Syntax

```
<Path>.getServices(Services:table)
```

Parameter

The parameter *Services* of data type *table* designates the name of an unformatted table (a table in a *Frame* or a local table) with three columns:

- Column 1 has the data type *string* and contains the **Name of the Service**.
- Column 2 has the data type *integer* and contains the **Priority**.
- Column 3 has the data type *integer* and contains the **Efficiency**.

Example

```
print .Resources.Worker:1.getServices(MyServicesTable)
```

SimTalk

[setServices \[SimTalk\] - Worker](#)

[Services \[SimTalk\] - Worker](#)

See also

[Services \[Worker\]](#)

goTo [SimTalk]

Sends the *Worker* designated by <Path> to the designated *Workplace*.

Remarks

If the *Worker* is interrupted, but must teleport to reach the intended destination, the *Worker* does not accept the order and the method returns false.

If the specified object is not a *Workplace* but a *work station*, the *Worker* walks to the closest *Workplace* assigned to the *work station* instead.

Type

Method

Syntax

```
<Path>.goTo(Workplace:object[, ReserveExclusively:boolean:=true]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Workplace* of data type *object* designates the *Workplace* to which you want to send the *Worker*.
- The optional parameter *ReserveExclusively* of data type *boolean* sets if the *Workplace* will be reserved exclusively for the *Worker* until he leaves the *Workplace* (true or no parameter). In this case the *Workplace* is not available any longer for another *Worker*, even while the designated *Worker* is on his way to the *Workplace*. If the *Workplace* is already occupied by another *Worker*, the *Worker* you sent stops and *goTo* returns false.

If you specify false, the *Workplace* is not going to be reserved. The *Worker* walks to the *Workplace*, and if the *Workplace* is occupied, he waits until it becomes available again.

Note

If *AutomaticMediation* is deactivated and *goTo* is called if the parameter *ReserveExclusively* is true, meaning that it reserves the *Workplace*, *Plant Simulation* checks if the *Workplace* still has available **Capacity**. If all places of the *Workplace* are either occupied or reserved, *Plant Simulation* shows an error message.

Default Value of the Parameter

The default value is true.

Return Value

The return value has the data type *boolean*.

false if the *Worker* is exporting a service at the moment. The *Worker* then stays where he is currently at.

Examples

```
.Resources.MyWorker:1.goTo(MyWorkplace)
```

```
.Resources.MyWorker:1.goTo(MyWorkplace, false)
```

SimTalk

[AutomaticMediation \[SimTalk\] - Worker](#)

[goToHome \[SimTalk\]](#)

[goToPool \[SimTalk\]](#)

[teleportTo \[SimTalk\]](#)

[teleportToPool \[SimTalk\]](#)

goToHome [SimTalk]

Sends the *Worker* designated by <Path> back to his *WorkerPool* or to his **Home Location** if you defined a **Home Location**.

Remarks

If the *Worker* is interrupted, but must teleport to reach the intended destination, the *Worker* does not accept the order and the method returns false.

Type

Method

Syntax

```
<Path>.goToHome → boolean
```

Return Value

The return value has the data type *boolean*.

false if the *Worker* is exporting a service at the moment. The *Worker* then stays where he is currently at.

Example

```
.Resources.MyWorker:1.goToHome
```

SimTalk

[goTo \[SimTalk\]](#)

[goToPool \[SimTalk\]](#)

[HomeLocation \[SimTalk\] - Worker](#)

[teleportTo \[SimTalk\]](#)

[teleportToPool \[SimTalk\]](#)

See also

[Home Location](#)

goToPool [SimTalk]

Sends the *Worker* designated by <Path> back to the *WorkerPool*.

Remarks

If the *Worker* is interrupted, but must teleport to reach the intended destination, the *Worker* does not accept the order and the method returns false.

Syntax

```
<Path>.goToPool → boolean
```

Return Value

The return value has the data type *boolean*.

false if the *Worker* is exporting a service at the moment. The *Worker* then stays where he is currently at.

Example

```
.Resources.MyWorker:1.goToPool
```

SimTalk

[goTo \[SimTalk\]](#)

[goToHome \[SimTalk\]](#)

[teleportTo \[SimTalk\]](#)

[teleportToHome \[SimTalk\]](#)

setServices [SimTalk] - Worker

Sets the services that the *Worker* designated by <Path> provides.

Type

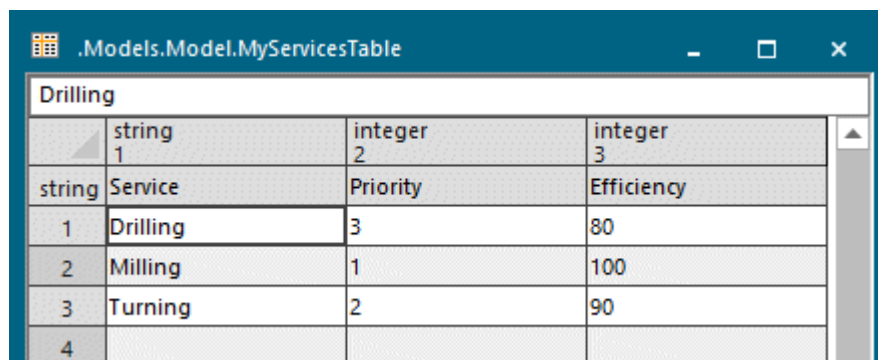
Method

Syntax

```
<Path>.setServices(Services:table)
```

Parameter

The parameter *Services* of data type *table* designates the name of an unformatted table (a *DataTable* in a *Frame* or a local table) with three columns:



	string 1	integer 2	integer 3
string	Service	Priority	Efficiency
1	Drilling	3	80
2	Milling	1	100
3	Turning	2	90
4			

- Column 1 has the data type *string* and contains the **Name of the Service**.
- Column 2 has the data type *integer* and contains the **Priority**.
- Column 3 has the data type *integer* and contains the **Efficiency**.

Example

```
.Resources.Worker:1.setServices(MyServicesTable)
.UserObjects.MyWorker.setServices(MyServicesTable)
```

SimTalk

getServices [SimTalk] - Worker

Services [SimTalk] - Worker

See also

[Services \[Worker\]](#)

teleportTo [SimTalk]

Teleports the *Worker* designated by <Path> to the designated *Workplace*.

Type

Method

Syntax

```
<Path>.teleportTo(Workplace:object) → boolean
```

Parameter

The parameter *Workplace* of data type *object* designates the *Workplace* to which you want to teleport the *Worker*.

Return Value

The return value has the data type *boolean*.

true if the *Worker* was teleported.

false if the *Worker* was not teleported because the *Worker* is already located on the *Workplace* for example.

Examples

```
.Resources.MyWorker:1.teleportTo(MyStation)  
-- returns true if a workplace at the station is available and the Worker  
can be teleported there
```

```
.Resources.MyWorker:2.teleportTo(MyWorkplace1)  
-- returns true if the Worker can be teleported to this Workplace
```

SimTalk

[teleportToHome \[SimTalk\]](#)

[teleportToPool \[SimTalk\]](#)

[goToHome \[SimTalk\]](#)[goToPool \[SimTalk\]](#)**teleportToHome [SimTalk]**

Teleports the *Worker* designated by <Path> back to his *WorkerPool* or to his *HomeLocation* if you defined a **Home Location**.

Type

Method

Syntax

```
<Path>.teleportToHome → boolean
```

Return Value

The return value has the data type *boolean*.

true if the *Worker* could be teleported to his **Home Location** or to his *WorkerPool*.

false if the *Worker* could not be teleported because he is already located at his **Home Location** or in his *WorkerPool* for example.

Example

```
.Resources.MyWorker:1.teleportToHome
```

SimTalk[teleportTo \[SimTalk\]](#)[teleportToPool \[SimTalk\]](#)[goTo \[SimTalk\]](#)[goToPool \[SimTalk\]](#)[HomeLocation \[SimTalk\] - Worker](#)

See also

[Home Location](#)

teleportToPool [SimTalk]

Teleports the *Worker* designated by <Path> back to the *WorkerPool*.

Syntax

```
<Path>.teleportToPool → boolean
```

Return Value

The return value has the data type *boolean*.

true if the *Worker* was teleported to his *WorkerPool*.

false if the *Worker* was not teleported to his *WorkerPool* because he was already located there for example.

Example

```
.Resources.MyWorker:1.teleportToPool
```

SimTalk

[teleportTo \[SimTalk\]](#)

[teleportToHome \[SimTalk\]](#)

[goTo \[SimTalk\]](#)

[goToHome \[SimTalk\]](#)

transportPart [SimTalk]

Makes the *Worker* designated by <Path> transport a single part from its current location to the specified destination. Transports several parts from the current location to their specified destination.

Remarks

Instead of one or several parts you can also specify a material flow object, which provides parts to be picked up. Then the *Worker* picks up all parts, which are ready to exit and carries them away when the *Worker* arrives at the specified *Station* until his **Capacity** is exhausted.

If no parts are ready to exit when the *Worker* arrives, *Plant Simulation* shows an error message.

If you do not specify a **Destination**, the part/the parts must already have a **Destination**. *Plant Simulation* saves the specified **Destination** in the part/the parts. You can query it with the attribute *Destination*.

Note

The *Worker* can only transport several parts if he can pick all parts up at the same location.

Type

Method

Syntax

```
<Path>.transportPart(PartOrStationOrParts:object/object[ ],
Destination:object]) -> void
<Path>.transportPart(Part:object[, Destination:object]) -> void
<Path>.transportPart(Parts:object[ ][, Destination:object]) -> void
<Path>.transportPart(Station:object[, Destination:object]) -> void
```

Parameters

You can specify the following parameters:

- The parameter *Part* of data type *object* designates the part that is to be transported.
- The parameter *Parts* of data type *object* designates an *array* of parts that are to be transported.
- The parameter *Station* of data type *object* designates the *Station* at which the *Worker* picks up the parts that are ready to exit and carries them away.

The *Station* can be any material flow object that provide parts to be picked up.

- The optional parameter *Destination* of data type *object* designates the *destination* object to which the *Worker* transports the part. If you do not specify the *destination*, you have to define a *destination* for each part to be transported.

Return Value

The method does not have a return value.

It outputs error messages, which might cause the method to fail, because:

- The specified *Worker* is a class object.
- The specified *Worker* is brokered automatically.
- The specified *Worker* cannot transport parts.
- The specified *Worker* already carries a part.
- The number of parts to be carried is greater than the current **Capacity** of the *Worker*.
- The part does not exist or does not have the correct object type.
- The **Destination** does not exist or does not have the correct object type.
- You did not specify a **Destination** and the part/the parts do not have a **Destination** of their own.
- The part itself, that is to be transported, has a defined **Destination**, which differs from the specified **Destination**.
- The parts to be transported have to be picked up at different locations.
- The part or one of the parts is already being transported by a *Worker*.
- No parts are to be picked up and carried away at the *Station* by the *Worker*.
- An error occurred while sending the *Worker* for picking up or transporting the parts.

Examples

```
waituntil .Resources.Worker:1.HasOrder = false  
.Resources.Worker:1.transportPart(Station.cont, Station1)
```

SimTalk

[cancelTransportPart \[SimTalk\]](#)

[Destination \[SimTalk\] - MUs](#)

Read-Only Attributes of the Worker

_Read-Only Attributes of the Worker

The *Worker* provides:

- The *read-only attributes* listed in the table of contents to the left.
- All *read-only attributes* of the *Exporter*, as the *Worker* is an *Exporter* with the capacity of 1.